Acceso a Datos

# Mapeo objeto-relacional

Tema 4

---

## 1. Concepto de mapeo objeto-relacional

Técnica de programación que se utiliza con el propósito de **convertir datos** entre los utilizados en un **lenguaje de programación orientado a objetos** y los utilizados en una **base de datos relacional**, gracias a la **persistencia**.

## 2. Herramientas ORM. Características

**ORM** Object Relational Mapping. Herramienta que transforma representaciones de datos de los Sistemas de Bases De Datos Relacionales a representaciones de objetos.

**Ventajas:**

- Facilitan el **mapeo de atributos** entre la BBDD y el modelo de objeto mediante archivos declarativos XML.
- Ayudan a **reducir el tiempo** de desarrollo.
- **Abstracción** de la base de datos.
- **Reutilización**.
- Permiten **persistir objetos** y generar el SQL correspondiente.
- **Lenguaje propio** para realizar las consultas.
- **Independencia** de la base de datos.
- Incentivan la **portabilidad y la escalabilidad.**

**Desventajas:**

- Su correcta utilización requiere **tiempo de aprendizaje.**
- Menor **rendimiento**.
- Sistemas **complejos**.

# 3. Hibernate

**Hibernate** Herramienta ORM para Java y .NET. Puede configurarse y ejecutarse en la mayoría de aplicaciones Java y entornos de desarrollo.

## 3.1. Arquitectura

Sigue una **arquitectura en capas** para mantenerlo aislado de tener que conocer las APIs subyacentes. Presenta las siguientes **interfaces**.

- **Configuration** Representa un **archivo de configuración o propiedades** requerido por Hibernate. Proporciona la conexión con la base de datos y la configuración del mapeo de clase.
- **Session Factory** Configura Hibernate para la aplicación que utiliza el archivo de configuración suministrado.
- **Session** Se utiliza para obtener una conexión física con una base de datos.
- **Transaction** Representa una unidad de trabajo con la base de datos.
- **Query** Objetos de consulta SQL utilizan el lenguaje HQL para recuperar datos de la base de datos y crear objetos.

## 3.2. Ficheros de configuración

El archivo **Hibernate.cfg.xml** contiene información sobre la conexión de base de datos, las asignaciones de recursos y otras propiedades de conexión. Sus propiedades más importantes son:

- **Hibernate.dialect** Lenguaje empleado.
- **Hibernate.connection.driver_class** Driver utilizado para la conexión.
- **Hibernate.connection.url** Dirección de la base de datos.
- **Hibernate.connection.username** Nombre del usuario, root por defecto.
- **Hibernate.connection.password** Contraseña de root.

El archivo **hibernate.reveng.xml** contiene los parámetros de conexión a la base de datos.

### 3.3. Ficheros de mapeo

Para cada clase que queramos persistir se creará un fichero XML con la información que permitirá mapear la clase a una base de datos relacional.

```
1  <?xml version="1.0"?>
2  <!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
3  "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
4  <hibernate-mapping>
5     <class name="Persistente.Departamentos" table="DEPARTAMENTOS" schema="ANTONIO">
6        <id name="deptNo" type="byte">
7           <column name="DEPT_NO" precision="2" scale="0" />
8           <generator class="assigned" />
9        </id>
10       <property name="dnombre" type="string">
11          <column name="DNOMBRE" length="15" />
12       </property>
13       <property name="loc" type="string">
14          <column name="LOC" length="15" />
15       </property>
16       <set name="empleadoses" inverse="true">
17          <key>
18             <column name="DEPT_NO" precision="2" scale="0" not-null="true" />
19          </key>
20          <one-to-many class="Persistente.Empleados" />
21       </set>
22    </class>
23 </hibernate-mapping>
```

## 4. Mapeo de colecciones, relaciones y herencia

### 4.1. Mapeo de colecciones

El elemento de mapeo de Hibernate utilizado para mapear una colección depende **del tipo de la interfaz.**

Las instancias de colección se distinguen por **la clave foránea de la entidad** que posee la colección. El elemento **mapea la columna clave de la colección.**

Las colecciones pueden contener **casi cuaquier tipo de datos**: esto implica que un objeto en una colección puede ser manejado con una semántica de valor, o podría ser una referencia a otra entidad con su propio ciclo de vida.

### 4.2. Mapeo de relaciones

Las relaciones usan las **transacciones**, puesto que los cambios pueden incluir varias tablas. Debe **respetarse la relación** en el modelo de objetos y en el modelo relacional.

Para mapear las relaciones se usan los **identificadores de objetos.**

### 4.3. Mapeo de herencia

Puede modelarse la jerarquía en una sola tabla, modelar la jerarquía completa en tablas, o mapear cada tabla en tablas concretas.

## 5. Clases persistentes

**Persistencia** Capacidad de los objetos para **guardarse y recuperarse** desde un medio de almacenamiento. Las clases persistentes representan **entidades de la base de datos.**

**Clase con capacidad de persistencia** Según el estándar **Java Data Objects**, aquella que implementa `javax.jdo.PersistenceCapable`. Esta inferfaz permite que los objetos pertenecientes a esas clases puedan **mantener su estado**, siempre que el estado esté compuesto por **atributos persistentes** que sean **independientes del contexto de ejecución.** Deben tener un constructor público sin argumentos, añadir un get/set por cada propiedad a persistir, e implementar el interfaz `Serializable`.

## 6. Sesiones: estados de un objeto

**Sesión** Objeto que representa **una unidad de trabajo con la base de datos.** Dispone de una API básica que permite **guardar y cambiar objetos**. Definido con la clase `SessionFactory`. Está formada por una **cola de sentencias SQL** y una **lista de objetos persistentes.**

Los estados en los que se puede encontrar un objeto son:

- **Transitorio** Objeto recién creado que **no ha sido enlazado** con el gestor de persistencia.
- **Persistente** Objeto **enlazado con la sesión**: todos los cambios serán persistentes.
- **Disociado** Objeto que **sigue en memoria** después de que termine la sesión.
- **Borrado** Objeto **marcado para ser borrado** de la base de datos.

# 7. Carga, almacenamiento y modificación de objetos

El método `Session.load` ofrece un mecanismo para capturar una instancia persistente. Acepta un objeto `Class` y **cargará el estado** de una nueva instancia de esa clase, inicializada en estado **persistente**.

Si no se está seguro de la **existencia de un objeto** en la base de datos, se debería usar `get`, que **devuelve un nulo** si no encuentra el objeto, en lugar de `load`, que tira una **excepción**.

Para **almacenar** objetos persistentes:

1. Se **instancia** un objeto nuevo.
2. Se obtiene una **sesión** y se comienza la **transacción.**
3. Se llama al método `save`, el cual **introduce el objeto en el contexto de persistencia** y devuelve el identificador del objeto.
4. Se realiza el **commit** de la transacción mediante el método `flush`.
5. Se cierra la sesión para **liberar el contexto** de persistencia.

`Session.delete` quita el estado de un objeto de la base de datos.

# 8. Consultas SQL

La **ejecución de SQL en Hibernate** se ejecuta mediante la interfaz SQLQuery, obtenida mediante el método `Session.createSQLQuery`:

```
1  sess.createSQLQuery("SELECT * FROM Personas").list();
2  sess.createSQLQuery("SELECT * FROM Personas).addEntity(Persona.class);
```

## 8.1. HQL: Hibernate Query Language

**HQL** Lenguaje de consulta usado por Hibernate, orientado a objetos e incluyendo conceptos de **herencia, polimorfismo y asociación**. Hibernate convertirá las consultas al SQL requerido por la base de datos.

Características:

- Soporte para **operaciones relacionales.**
- Usa **clases y atributos** en lugar de tablas y columnas.

- Usa los **tipos de datos** de Java.
- Permite tratar con las **colecciones de Java**.
- Permite **navegar entre los distintos objetos** en la propia consulta.
- Ofrece sus resultados **en forma de objetos**.
- Soporte para **consultas polimórficas**.
- **Independiente del sistema** de bases de datos que se use.

La sintaxis es [**select**] **from** [**where**] [groupby] [**having**] [orderby], donde solo `from` es obligatorio.

```
1  SELECT c FROM Ciclo c ORDER BY nombre
2  SELECT p FROM Profesor p WHERE nombre="ISABEL" AND apel<>"ORELLANA"
```

### 8.1.1. Consultas por criterios

El objeto `Query` da acceso a todas las funcionalidades para poder **leer objetos** desde la base de datos.

```
1  Query query = session.createQuery("SELECT p FROM Profesor p");
2  List<Profesor> profesores = query.list();
3  for (Profesor profesor : profesores)
4  {
5      System.out.println(profesor.toString());
6  }
```

El método `list` devuelve una **lista con todos los objetos** devueltos por la consulta.

```
1  Query query = session.createQuery("SELECT p FROM Profesor p");
2  List<Profesor> profesores = query.list();
3  for (Profesor profesor : profesores) {
4      System.out.println(profesor.toString());
5  }
```

`uniqueResult` devuelve el **primer o único objeto** que devuelva una consulta.

```
1  Profesor profesor = (Profesor) session.createQuery("SELECT p FROM Profesor p
2  WHERE id=1001").uniqueResult();
3  System.out.println("Profesor con Id 1001=" + profesor.getNombre());
```

Los métodos `setMaxResults` y `setFirstResult` permiten establecer **paginación.**

```
1  int tamanyoPagina = 10;
2  int paginaAMostrar = 7;
3  Query query = session.createQuery("SELECT p FROM Profesor p Order By p.id");
4  query.setMaxResults(tamanyoPagina);
5  query.setFirstResult(paginaAMostrar * tamanyoPagina);
6  List<Profesor> profesores = query.list();
7  for (Profesor profesor : profesores) {
8      System.out.println(profesor.toString());
9  }
```

La anotación `@NamedQuery` en un fichero de mapeo de Hibernate permite incluir las consultas **en un fichero externo** para poder cargarlas a posteriori mediante `getNamedQuery`.

```
1  @Entity
2  @NamedQueries({
3  @NamedQuery(name="Profesores.findAll", query="SELECT p FROM Profesor p")
4  @NamedQuery(name="Profesores.ByName", query="SELECT p FROM Profesor p
5  where p.nombre =:nombre and p.ape1=:ape1 and p.ape2=:ape2")
6  })
7  public class Profesor {
8  ...
9  }
10
11 [...]
12
13 Query query = session.getNamedQuery("Profesores.findAll");
14 List<Profesor> profesores = query.list();
15 for (Profesor profesor : profesores) {
16  System.out.println(profesor.toString());
17 }
```

# 9. Gestión de transacciones

Se pueden realizar **transacciones** mediante el método `beginTransaction`:

```
1  Session session = HibernateUtil.getSessionFactory().openSession();
2  Transaction tx = null;
3  try
4  {
5      tx = session.beginTransaction();
6  // Utilizar la Session para saveOrUpdate/get/delete/...tx.commit();
7  } catch (Exception e) {
8      if (tx != null) {
9          tx.rollback();
10         throw e;
11     }
12 } finally {
13 session.close();
14 }// Al finalizar la aplicacion ...HibernateUtil.shutdown( );
```

© 2026 - Esteban N. - estnicolas@proton.me

Este texto se encuentra licenciado bajo Creative Commons Atribución-Reconocimiento/NoComercial-CompartirIgual 4.0 Internacional.
Para ver una copia de esta licencia visite:
https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.es.




Ninguna parte de este documento, código o texto, ha sido realizada, apoyada, consultada, generada, corregida o involucrada en ninguna forma o contexto con inteligencia artificial, ni lo será en cualquier futura posible versión o corrección.
https://notbyai.fyi